

Chapter 3: Types & Operations

By Irina Galata

Now that you know how to perform basic operations and manipulate data using operations, it's time to learn more about **types**. Formally, a **type** describes a set of values and the operations that can be performed on them.

In this chapter, you'll learn about handling different types, including strings which allow you to represent text. You'll learn about converting between types and you'll also be introduced to type inference which makes your life as a programmer a lot simpler.

You'll also learn about `Pair` and `Triple`, which allow you to make your own types made up of two or three values of any type, respectively.

Finally, you'll learn about the important types `Any`, `Unit`, and `Nothing`.

Type conversion

Sometimes you'll have data in one format and need to convert it to another. The naïve way to attempt this would be like so:

```
var integer: Int = 100
var decimal: Double = 12.5
integer = decimal
```

Kotlin will complain if you try to do this and will spit out the following error:

```
Type mismatch.
Required: Int
Found:     Double
```

Some programming languages aren't as strict and will perform conversions like this automatically. Experience shows this kind of automatic conversion is a source of software bugs and often hurts performance.

Kotlin disallows you from assigning a value of one type to another and avoids these issues.

Remember, computers rely on us programmers to tell them what to do. In Kotlin, that includes being explicit about type conversions. If you want the conversion to happen, you have to say so!

Instead of simply assigning, you need to explicitly say that you want to convert the type. You do it like so:

```
integer = decimal.toInt()
```

The assignment now tells Kotlin unequivocally that you want to convert from the original type, `Double`, to the new type, `Int`.

Note: In this case, assigning the decimal value to the integer results in a loss of precision: The `integer` variable ends up with the value 12 instead of 12.5. This is why it's important to be explicit. Kotlin wants to make sure you know what you're doing and that you may end up losing data by performing the type conversion.

Operators with mixed types

So far, you've only seen operators acting independently on integers or doubles. But what if you have an integer that you want to multiply by a double?

You might think you have to do something like this:

```
val hourlyRate: Double = 19.5
val hoursWorked: Int = 10
val totalCost: Double = hourlyRate * hoursWorked.toDouble()
```

In this example, `hoursWorked` is explicitly converted to a `Double`, to match the type of `hourlyRate`. But, it turns out, this is unnecessary. Kotlin will allow you to multiply these values without any conversion, like so:

```
val totalCost: Double = hourlyRate * hoursWorked
```

In Kotlin, you can apply the `*` operator to mixed types. This rule also applies to the other arithmetic operators. Even though `hoursWorked` is an `Int`, this will not remove the precision of `hourlyRate`. The result will still be `195.0`! Kotlin attempts to be as concise as possible as often as possible and will try to infer the expected behavior when it can. This way you spend less time worrying about types and more time building awesome things!

Type inference

Up to this point in this book, each time you've seen a variable or constant declared it's been accompanied by a type declaration. You may be asking yourself why you need to bother writing the `: Int` and `: Double`, since the right hand side of the assignment *is already* an `Int` or a `Double`. It's redundant, to be sure; your clever brain can see this without too much work.

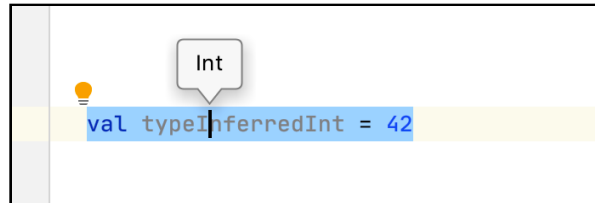
It turns out the Kotlin compiler can deduce this as well. It doesn't need you to tell it the type all the time — it can figure it out on its own. This is done through a process called **type inference**. Not all programming languages have this, but Kotlin does, and it's a key component of Kotlin's power as a language.

So, you can simply drop the type in most places where you see one.

For example, consider the following constant declaration:

```
val typeInferredInt = 42
```

Sometimes it's useful to check the inferred type of a variable or constant. You can do this in IntelliJ by clicking on the variable or constant's name and holding down the **Control** + **Shift** + **P** keys. IntelliJ will display a popover like this:

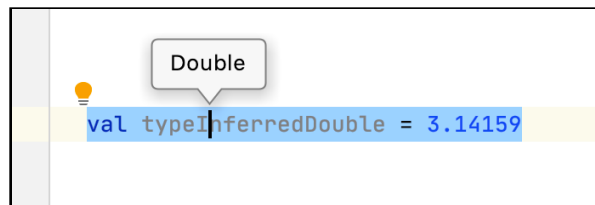


IntelliJ tells you the inferred type by giving you the declaration you would have had to use if there were no type inference. In this case, the type is `Int`.

It works for other types, too:

```
val typeInferredDouble = 3.14159
```

Holding down the **Control** + **Shift** + **P** keys reveals the following:



You can see from this that type inference isn't magic. Kotlin is simply doing what your brain does very easily. Programming languages that don't use type inference can often feel verbose, because you need to specify the often obvious type each time you declare a variable or constant.

Note: In later chapters, you'll learn about more complex types where sometimes Kotlin can't infer the type. That's a pretty rare case though, and you'll see type inference used for most of the code examples in this book — except in cases where you want to highlight the type.

Sometimes you want to define a constant or variable and ensure it's a certain type, even though what you're assigning to it is a different type. You saw earlier how you can convert from one type to another. For example, consider the following:

```
val wantADouble = 3
```

Here, Kotlin infers the type of `wantADouble` as `Int`. But what if you wanted `Double` instead? The first thing you could do is the following:

```
val actuallyDouble = 3.toDouble()
```

This is like you saw before with type conversion.

Another option would be to not use type inference at all and do the following:

```
val actuallyDouble: Double = 3.0
```

Something you may be tempted to try is this:

```
val actuallyDouble: Double = 3
```

This is not allowed and results in the compiler giving you the following error: The integer literal does not conform to the expected type `Double`.

Note: In Kotlin literal values like `3` have a specific type, `Int`. If you want to convert them to a different type, you must do so explicitly, by calling `toDouble()` for example. A literal number value that contains a decimal point can only be used as a `Double` and must be explicitly converted if you want to use it as something else. This is why you're not allowed to assign the value `3` to constant `actuallyDouble`.

Likewise, literal number values that *do* contain a decimal point cannot be integers. This means you could have avoided this entire discussion had you started with:

```
val wantADouble = 3.0
```

Sorry!

Mini-exercises

1. Create a constant called `age1` and set it equal to 42. Create a constant called `age2` and set it equal to 21. Check using `Control+Shift+P` that the type for both has been inferred correctly as `Int`.
2. Create a constant called `avg1` and set it equal to the average of `age1` and `age2` using the naive operation $(age1 + age2) / 2$. Use `Control+Shift+P` to check the type and check the result of `avg1`. Why is it wrong?
3. Correct the mistake in the above exercise by converting `age1` and `age2` to type `Double` in the formula. Use `Control+Shift+P` to check the type and check the result of `avg1`. Why is it now correct?

Strings

Numbers are essential in programming, but they aren't the only type of data you need to work with in your apps. Text is also an extremely common data type, such as people's names, their addresses, or even the words of a book. All of these are examples of text that an app might need to handle.

Most computer programming languages store text in a data type called a **string**. This chapter introduces you to strings, first by giving you background on the concept of strings and then by showing you how to use them in Kotlin.

How computers represent strings

Computers think of strings as a collection of individual **characters**. In Chapter 1 of this book, you learned that numbers are the language of CPUs, and all code, in whatever programming language, can be reduced to raw numbers. Strings are no different!

That may sound very strange. How can characters be numbers? At its base, a computer needs to be able to translate a character into the computer's own language, and it does so by assigning each character a different number. This forms a two-way mapping from character to number that is called a **character set**.

When you press a character key on your keyboard, you are actually communicating the number of the character to the computer. Your word processor application converts that number into a picture of the character and finally, presents that picture to you.

Unicode

In isolation, a computer is free to choose whatever character set mapping it likes. If the computer wants the letter **a** to equal the number 10, then so be it. But when computers start talking to each other, they need to use a common character set. If two computers used different character sets, then when one computer transferred a string to the other, they would end up thinking the strings contained different characters.

There have been several standards over the years, but the most modern standard is **Unicode**. It defines the character set mapping that almost all computers use today.

Note: You can read more about Unicode at its official website, <http://unicode.org/>.

As an example, consider the word **cafe**. The Unicode standard tells us that the letters of this word should be mapped to numbers like so:

c	a	f	e
99	97	102	101

The number associated with each character is called a **code point**. So in the example above, **c** uses code point 99, **a** uses code point 97, and so on.

Of course, Unicode is not just for the simple Latin characters used in English, such as **c**, **a**, **f** and **e**. It also lets you map characters from languages around the world. The word **cafe**, as you're probably aware, is derived from French, in which it's written as **café**. Unicode maps these characters like so:

c	a	f	é
99	97	102	233

And here's an example using Chinese characters (this, according to Google translate, means "Computer Programming"):

电	脑	编	程
30005	33041	32534	31243

You’ve probably heard of emojis, which are small pictures you can use in your text. These pictures are, in fact, just normal characters and are also mapped by Unicode. For example:

	
128169	128512

This is only two characters. The code points for these are very large numbers, but each is still only a single code point. The computer considers these as no different than any other two characters.

Note: The word “emoji” comes from Japanese, where “e” means picture and “moji” means character.

Strings in Kotlin

Kotlin, like any good programming language, can work directly with characters and strings. It does so through the data types `Char` and `String`, respectively. In this section, you’ll learn about these data types and how to work with them.

Characters and strings

The `Char` data type can store a single character which must be wrapped in single quotes. For example:

```
val characterA: Char = 'a'
```

This data type is designed to hold only single characters. The `String` data type, on the other hand, stores multiple characters, which must be wrapped in double quotes. For example:

```
val stringDog: String = "Dog"
```

It’s as simple as that! The right-hand side of this expression is what’s known as a **string literal**; it’s the Kotlin syntax for representing a string.

Of course, type inference applies here as well. If you remove the type in the above declaration, then Kotlin does the right thing and makes the `stringDog` a `String` constant:

```
val stringDog = "Dog" // Inferred to be of type String
```

Concatenation

You can do much more than create simple strings. Sometimes you need to manipulate a string, and one common way to do so is to combine it with another string.

In Kotlin, you do this in a rather simple way: by using the addition operator. Just as you can add numbers, you can add strings:

```
var message = "Hello" + " my name is "  
val name = "Joe"  
message += name // "Hello my name is Joe"
```

You need to declare `message` as a variable rather than a constant because you want to modify it. You can add string literals together, as in the first line, and you can add string variables or constants together, as in the last line.

It's also possible to add characters directly to a string. This is similar to how you can easily work with numbers if one is an `Int` and the other is a `Double`.

To add a character to a string, you do this:

```
val exclamationMark: Char = '!'  
message += exclamationMark // "Hello my name is Joe!"
```

No need to explicitly convert the `Character` to a `String` before you add it to `message`; Kotlin takes care of that for you!

String templates

You can also build up a string by using **string templates**, which use a special Kotlin syntax that lets you build a string in a way that's easy to read:

```
message = "Hello my name is $name!" // "Hello my name is Joe!"
```

This is much more readable than the example from the previous section. It's an extension of the string literal syntax, whereby you replace certain parts of the string with other values. Simply prepend the value you want to insert with a `$` symbol.

This syntax works in the same way to build a string from other data types, such as numbers:

```
val oneThird = 1.0 / 3.0
val oneThirdLongString = "One third is $oneThird as a decimal."
```

Here, you use a Double in the template. At the end of this code, your oneThirdLongString constant will contain the following:

```
One third is 0.3333333333333333 as a decimal.
```

Of course, it would actually take infinite characters to represent one third as a decimal, because it's a repeating decimal. Using string templates with a Double gives you no way to control the precision of the resulting string.

This is an unfortunate consequence of using string templates: they're simple to use, but offers no ability to customize the output.

You can also put expressions inside a string template, by following the \$ symbol with a pair of braces that contain the expression:

```
val oneThirdLongString = "One third is ${1.0 / 3.0} as a decimal."
```

The result is just the same as before.

Multi-line strings

Kotlin has a neat way to express strings that contain multiple lines. This can be rather useful when you need to put a very long string in your code.

You do it like so:

```
val bigString = """
    You can have a string
    that contains multiple
    lines
    by
    doing this.
    """.trimMargin()
println(bigString)
```

The three double-quotes signify that this is a multi-line string. Handily, the first and final new lines do not become part of the string. This makes it more flexible as you don't have to have the three double-quotes on the same line as the string.

In the case above, it will print the following:

```
You can have a string
that contains multiple
lines
by
doing this.
```

Notice `|`, also known as the “pipe character”, at the start of each line as well as the call to `trimMargin()`. This prevents the string from having leading spaces, allowing you to format your code with pretty indentation without affecting the output.

Mini-exercises

1. Create a string constant called `firstName` and initialize it to your first name. Also create a string constant called `lastName` and initialize it to your last name.
2. Create a string constant called `fullName` by adding the `firstName` and `lastName` constants together, separated by a space.
3. Using string templates, create a string constant called `myDetails` that uses the `fullName` constant to create a string introducing yourself. For example, it could read: "Hello, my name is Joe Howard."

Pairs and Triples

Sometimes data comes in groups. An example of this is a pair of (x, y) coordinates on a 2D grid. Similarly, a set of coordinates on a 3D grid is comprised of an x-value, a y-value and a z-value. In Kotlin, you can represent such related data in a very simple way through the use of a `Pair` or `Triple`.

Other languages use a type named “Tuple” to hold similar combinations of values.

`Pair` or `Triple` are types that represent data composed of two or three values of any type. If you want to have more than three values, you use what Kotlin calls a *data class*, which you will cover in a future chapter.

Sticking with `Pair` for now, as an example you can define a pair of 2D coordinates where each axis value is an integer, like so:

```
val coordinates: Pair<Int, Int> = Pair(2, 3)
```

The type of `coordinates` is `Pair<Int, Int>`. The types of the values within the `Pair`, in this case `Int`, are separated by commas and surrounded by `<>`. The code for creating the `Pair` is much the same, with each value separated by commas and surrounded by parentheses.

Type inference can infer `Pair` types too:

```
val coordinatesInferred = Pair(2, 3)
```

You can make this even more concise, which Kotlin loves to do, by using the `to` operator:

```
val coordinatesWithTo = 2 to 3
```

You could similarly create a `Pair` of `Double` values, like so:

```
val coordinatesDoubles = Pair(2.1, 3.5)  
// Inferred to be of type Pair<Double, Double>
```

Or you could mix and match the types comprising the pair, like so:

```
val coordinatesMixed = Pair(2.1, 3)  
// Inferred to be of type Pair<Double, Int>
```

And here's how to access the data inside a `Pair`:

```
val x1 = coordinates.first  
val y1 = coordinates.second
```

You can reference each item in the `Pair` by its position in the pair, starting with `first`. So in this example, `x1` will equal 2 and `y1` will equal 3.

In the previous example, it may not be immediately obvious that the first value is the x-coordinate and the second value is the y-coordinate. This is another demonstration of why it's important to *always* name your variables in a way that avoids confusion.

Fortunately, Kotlin allows you to use a *destructuring declaration* on individual parts of a `Pair`, and you can be explicit about what each part represents. For example:

```
val (x, y) = coordinates
```

```
// x and y both inferred to be of type Int
```

Here, you extract the values from `coordinates` and assign them to `x` and `y`.

`Triple` works much the same way as `Pair`, just with three values instead of two.

If you want to access multiple parts of a `Triple` at the same time, as in the examples above, you can also use a shorthand syntax to make it easier:

```
val coordinates3D = Triple(2, 3, 1)
val (x3, y3, z3) = coordinates3D
```

This declares three new constants, `x3`, `y3` and `z3`, and assigns each part of the `Triple` to them in turn. The code is equivalent to the following:

```
val coordinates3D = Triple(2, 3, 1)
val x3 = coordinates3D.first
val y3 = coordinates3D.second
val z3 = coordinates3D.third
```

If you want to ignore a certain element of a `Pair` or `Triple`, you can replace the corresponding part of the declaration with an underscore. For example, if you were performing a 2D calculation and wanted to ignore the `z`-coordinate of `coordinates3D`, then you'd write the following:

```
val (x4, y4, _) = coordinates3D
```

This line of code only declares `x4` and `y4`. The `_` is special and simply means you're ignoring this part for now.

Mini-exercises

1. Declare a constant `Triple` that contains three `Int` values. Use this to represent a date (month, day, year).
2. Extract the values in the triple into three constants named `month`, `day` and `year`.
3. In one line, read the month and year values into two constants. You'll need to employ the underscore to ignore the day.
4. Since the values inside `Pairs` and `Triples` cannot be modified, you will need to extract the values from them, make any modifications you want, and then create a new `Pair` or `Triple`. Using the values you extracted in step three, modify the month value and create a new `Pair` containing the modified month along with the unmodified year.

Number types

Many C-based languages like Java have primitive types that take up a specific number of bytes. For example, in Java, a 32-bit signed primitive number is an `int`. There is also an object version of an `int` known as an `Integer`. You may be wondering why it is necessary to have two types that store the same number type.

Well, primitives require less memory, which means they are better for performance, but they also lack some of the features of `Integer`. The good news is in Kotlin, you don't have to worry about whether you need to use a primitive type or an object type. Kotlin handles that complexity for you, so all you have to do is use an `Int`.

You've been using `Int` to represent whole numbers which are represented using 32 bits. Kotlin provides many more number types that use different amounts of storage. For whole numbers, you can use `Byte`, `Short`, and `Long`. These types consume 1, 2, and 8 bytes of storage respectively. Each of these types use one bit to represent the sign.

Here is a summary of the different integer types and their storage size in bytes. Most of the time you will just want to use an `Int`. These become useful if your code is interacting with another piece of software that uses one of these more exact sizes or if you need to optimize for storage size.

Type	Minimum value	Maximum value	Storage size
Byte	-128	127	1
Short	-32768	32767	2
Int	-2147483648	2147483647	4
Long	-9223372036854775807	9223372036854775806	8

You've been using `Double` to represent fractional numbers. Kotlin offers a `Float` type which has less range and precision than `Double`, but requires half as much storage. Modern hardware has been optimized for `Double` so it is the one that you should reach for unless you have good reason not to.

Type	Minimum value	Maximum value	Precision	Storage size
Float	-3.4028235E+38	3.4028235E+38	6 digits	1
Double	-1.797693E+308	1.797693E+308	15 digits	2

Most of the time you will just use `Int` and `Double` to represent numbers, but every once in a while, you might encounter the other types. Suppose you need to add together a `Short` with a `Byte` and a `Long`. You can do that like so:

```
val a: Short = 12
val b: Byte = 120
val c: Int = -100000

val answer = a + b + c // Answer will be an Int
```

Any, Unit, and Nothing Types

The `Any` type can be thought of as the mother of all other types (except nullable types, which will be covered in Chapter 7). Every type in Kotlin, whether an `Int` or a `String`, is also considered an `Any`. This is similar to the `Object` type in Java, which is the root of all types except primitives.

For example, it is perfectly valid Kotlin to declare an `Int` literal and `String` literal as `Any` like so:

```
val anyNumber: Any = 42
val anyString: Any = "42"
```

`Unit` is a special type which only ever represents one value: the `Unit` object. It is similar to the `void` type in Java, except it makes working with generics easier (which will be covered in Chapter 18). Every function (you will cover functions in Chapter 6, but for now think of a function as a piece of reusable code) which does not explicitly return a type, e.g., a `String`, returns `Unit`.

For example, here is a function that simply adds `2 + 2` and prints the result but does not actually return anything:

```
fun add() {
    val result = 2 + 2
    println(result)
}
```

The return type `Unit` is implied, so the above function is the same as this:

```
fun add(): Unit {  
    val result = 2 + 2  
    println(result)  
}
```

Nothing is a type that is helpful for declaring that a function not only doesn't return anything, but also never completes.

This can occur if a function either causes the program to stop completely by throwing an `Exception` or if it simply goes on forever without ever finishing.

By way of example, the saddest function ever written:

```
fun doNothingForever(): Nothing {  
    while(true) {  
  
    }  
}
```

You will cover while loops more in the next chapter, but for now understand that this function will simply run forever without ever returning anything. Welcome to the land of Nothing!

Challenges

1. Create a constant called `coordinates` and assign a pair containing two and three to it.
2. Extract the values 2 and 3 from `coordinates` into two variables called `row` and `column`.
3. Which of the following are valid statements?

```
val character: Character = "Dog"  
val character: Char = 'd'  
val string: String = "Dog"  
val string: String = 'd'
```


4. Is this valid code?

```
val date = Triple(15, 8, 2015)
val day = date.First
```

5. What is wrong with the following code?

```
val name = "Joe"
name += " Howard"
```

6. What is the type of the constant named value?

```
val triple = Triple(100, 1.5, 10)
val value = triple.second
```

7. What is the value of the constant named month?

```
val newDate = Triple(15, 8, 2015)
val month = newDate.second
```

8. What is the value of the constant named summary?

```
val number = 10
val multiplier = 5
val summary = "$number multiplied by $multiplier equals ${number * multiplier}"
```

9. What is the sum of a and b, minus c? What is its type?

```
val a = 4
val b: Short = 100
val c: Byte = 12
```

10. What is the numeric difference between `kotlin.math.PI` and `kotlin.math.PI.toFloat()`?

Key points

- Type conversion allows you to convert values of one type into another.
- Kotlin will convert types for you when using an operator, such as the basic arithmetic operators (+, -, *, /), with mixed types.
- Type inference allows you to omit the type when Kotlin already knows it.
- **Unicode** is the standard for mapping characters to numbers.
- A single mapping in Unicode is called a **code point**.
- The `Character` data type stores single characters. The `String` data type stores collections of characters, or strings.
- You can combine strings by using the addition operator.
- You can use **string templates** to build a string in-place.
- You can use `Pairs` and `Triples` to group data into a single data type.
- There are many kinds of numeric types with different storage and precision capabilities.
- `Any` is the mother of all non-nullable types, `Unit` is kind of like `void` in Java, and `Nothing` is well, nothing.

Where to go from here?

You can find all the code from this chapter, along with solutions to the mini-exercises and challenges, in the materials for this chapter.

Types are a fundamental part of programming. They're what allow you to correctly store your data. You've seen a few more here, including strings, pairs, and triples as well as a bunch of numeric types. Later on in the book you'll learn how to define your own types with **classes**, **enums** and **interfaces**.

In the next chapter, you'll learn about Boolean logic and simple control flow. This is required for any program to be able to make decisions about how the program should proceed based on the data it's manipulating.